• DP (small I/P space) – overlapping subproblems and optimal substructure → DP [large → recursion]

Input drawn from small input space and generally seek to recursively divide the input by some constant factor at each iteration. (Divide & conquer)

shallow recursion trees; polynomial runtime

[recursion
constant subtraction : DP. → deep recursion trees (exponentially w.r.t. D&C Trees)

• DP → bottom up
greedy → top down

Greedy : rule is selected → ↓ i/p size
→ select / reject some portion of i/p.
→ ~~not need dp~~.

Best available choice at any moment
↓
Best choice overall

designed to mirror the overall solution.

(doesn't depend on future choices) or sol^n to subproblems.

[DP :- Solves subproblems before making a choice unlike greedy - makes its first choice before solving any subproblem. → Local optimality implying global optimality.

• Greedy algo - easy to state but more difficult to prove correct/optimal.

[0/1 Knapsack → ✗ Greedy]
But fractional Knapsack → ✓ → Calculate value density v_i/w_i per item. → sort ascending order →

Greedy Choice: Take as much of the item with the highest remaining value density as we can fit in our bag.

• Dijkstra: single source shortest path. (non-negative edge wts)
↳ min priority queue - where priority is set to be computed distance from s.
- inserting a value ($d(u)=\infty$)
- extracting the min-value. (extract-Min)
- while $Q \neq \{\}$ do updating distance (Change-Key) ↓ $\pi(v)=u$

each needs $O(\log n)$ time

(lines 11-13: As with BFS/DFS, they are separate loop. (indep. of lines 6 loop) for while (for))

$O(m)$

sol^n to larger problems rely exclusively + (1) recursively on smaller subproblems.

MST (Min^m spanning tree) such that sum of all weights in tree is minimized.

• Prim's Algo → undirected graphs only
→ can have -ve edge weights.

(for dijkstra's algo - diff^t starting nodes can produce diff^t cost spanning trees)

[Difference b/w Prim's & Dijkstra's :→ starting node s is not fundamental; comparison against & value reassignment is not cumulative.

Unweighted Interval Scheduling :- If two intervals overlap in time, then they conflict and cannot both be scheduled. Goal :- choose a set of intervals which are mutually compatible.

[size of set is maximized over all possible non-conflicting sets]

• Proving that greedy algo is applicable
→ Proving feasibility - does algo actually produce a correct solution (easier to prove) [no worse than → comparable]
→ Proving optimality - (harder to prove)
↳ Is the algorithm comparable to any other sol^n w.r.t. its objective (will algo - always choose max^m non-overlapping set.

(1) Earliest available starting time ✗ optimal
(2) smallest (shortest) available interval

• Not valid algo :- fails to prove valid sol^n for some input.

(3) Fewest no. of conflicts. → ✗
(4) Earliest finishing time → [optimal]
[$s(i) < s(j) < f(i)$ or $s(i) < f(j) < f(i)$]

Prove: local optimal decisions lead to overall globally optimal algorithm.

[Greedy Algorithm stays Ahead]

→ we assume an optimal algo. O & assume that our algo always does at least as good as O.

• Comparison based sorting : (logn) time

Deadline Interval Scheduling → ~~Great~~ decision by deadline $d(i_k)$, needs to run continuously for $t(i_k)$ units. (Tasks - unweighted & all tasks are scheduled, yet still do not overlap.

→ We'll compute start & finish times based on the interval we'll assign each task.

→ Assumption : Always - some chance that some tasks will run past their deadline.

→ lateness $l(i_k) = \max\{0, \underbrace{f(i_k)}_{\text{finish time}} - d(i_k)\}$ (No negative lateness)

Task: Minimizing max^m lateness.

→ slack time, $\text{slack}(i_k) = \underbrace{d(i_k)}_{\text{deadline } d} - \underbrace{t(i_k)}_{\text{time entering}}$

[how much leeway we have to push a job back before its late]

→ Idle time $(s(i_{k+1}) - f(i_k))$ → gap in b/w end of one job and beginning of the next.

① Shortest time interval. ×

② Smallest slack time. ×

③ Earliest remaining deadline first } optimal soln.

• deadline interval scheduling → will schedule all jobs regardless of lateness.

- DP weights & no. of jobs finished before it,
  - overlapping subproblems; Optimal substructure
- $T(n) = aT(n/b) + f(n)$: definitely polynomial runtime if $f(n)$ is polynomial.

Top down approach

- Memoization: Once we compute one of the subproblems - we can store it for later use.
- optimal sol^n to WIS maynot be one with maximal no. of intervals.
- For any interval, we can split problem in 2 cases < 0 choices interval / 0 doesn't
- optimal substructure - we can frame problem recursively in making above choice along with result of O/P of 0 on strictly smaller subproblem.
- WIS :→ $O(i) = \max\{O(i-1), w_i + O(p(i))\}$
- fun (WIS): → $O(2^n)$

  if $i=0$ return 0

  else return max{ WIS-basic $(i-1)$, w_i + WIS basic$(p(i))$ }

  ↳ Assumptions
  - DS → start & finish time
  - Intervals sorted - ascending order
  - we have access to p/ compute it in initializ. or
  - ↳ we have access to p/ compute it in initializ.

⇓

function WIS_memo. (M, i):

if $i=0$ then return 0; if $M[i] \neq -1$ then return $M[i]$

else return $M[i] = \max\{$ WIS_memo $(i-1)$, w_i + WIS-memo $(p(i))\}$

return $M[i]$

↳ Assumption
- M: externally allocated
- we've sol^n s to all subproblems of $O(n)$

↳ $\boxed{O(n)}$ (M fills exactly after $(n-1)$ recursion calls)

- Subset sum problem: (n WIS - with potential choice of an interval, we could automatically eliminate a no. of other intervals. But here, we don't have luxury of reducing our search space in such manner.
- We need to consider subproblems which vary on both
  - → smaller choices of subset → smaller possible total wts
  - → $O(nW)$ time (0-1 Knapsack) O/P → 2D array
- Provided the subproblems are polynomial in size & quantity → D.P sol^n will be polynomial in complexity
- $O(n)$ → Recovering choices made in memoization (WIS/SS)
- 0/1 Knapsack: We sum on value & use weight as an upperbound.

Divide: Recursively split problem into some no. of smaller subproblems

Conquer: When I/P size is below constant threshold - use any known technique to solve the problem. Combine - while returning from recursion combine the child branches.

→ Each call to merge sort needs $\Theta(n)$ oper^{n}s

→ Each level of binary tree: 2^i nodes.

→ I/P size at each function call: $n/2^i$ (as merge → 2 splits)

→ Overall work: Sum of work at individual level & 2^i instances requiring $(c(n/2^i))$ work each. [cn work - each level]

→ $\log n$ levels → $cn \log n$.

$$T(n) = \begin{cases} c_0 & n \leq n_0 \\ aT(n/b) + f(n) & n > n_0 \end{cases}$$

$\Theta(n)$ a = no. of recursive calls. b = splits

Recurrence of above form generate a-ary trees

input size n/b^i

- $2^{\log_2(n)} = n^{\log_2(n)}$

$T(n) = \Theta(n^{\log_b a})$ $(a > b)$ $a > 2$

$a = 1$ $T(n) = O(n)$ $a = 2$ → $T(n)$ $\Theta(n \log n)$

- If $f(n)$ is faster than $\Theta(n^{\log_b a})$ then $T(n) = \Theta(n^{\log_b a})$
- $f(n)$ same as $\Theta(n^{\log_b a}) \Rightarrow T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$
- $f(n)$ slower : $T(n) = \Theta(f(n))$

Master Theorem

$\epsilon > 0$; $f(n) = O(n^{\log_b a - \epsilon}) \rightarrow T(n) = \Theta(n^{\log_b a})$

$k \geq 0$; $f(n) = \Theta(n^{\log_b a} \log^k n) \rightarrow T(n) =$

$\epsilon > 0$ & $f(n) = \Omega(n^{\log_b a + \epsilon}) \rightarrow a f(n/b) \leq c f(n)$.

$T(n) = \Theta(f(n))$

- Naive Recursive Multiplication → $a=4, b=2$; $O(n^2)$
- Recursive → $a=3, b=2$ → $O(n^{1.59})$ $\boxed{2^a \text{ nodes}}$.
- Substitution method → K useful → X Runtime
- Stopping recursion early and using an asymptotically worse algo as base case won't necessarily negatively impact the overall asymptotic runtime of algo.
- $T(n) = T(n-2) + \Theta(1) = T(n-2k) + k\Theta(1)$

  $T(0)$ → if $k = n/2$
- Using ceiling of $n/2$ instead of floor of $n/2$ in recursive multi - will correctly calculate product of x, y.

- Simple path - which doesn't visit same node twice
- Adjacency Matrix < Neighbour Testing $\Theta(1)$ (freq add/delete edges) / Listing $\Theta(n)$.
- Adj. List < N. Testing → $O(\deg V)$, $O(n)$, $\Omega(1)$ / N. Listing $(\Theta(\deg V)$, $O(n)$, $\Omega(1)$
- It is possible for algo runtime to run $\Theta(m+n)$ with adj list but best possible bound using adj. matrix is $\Theta(n^2) \Rightarrow \Omega(n^2)$
- Spanning tree: smallest subset of edges which path connects all nodes in a connected component, though is not generally unique.
- BFS - Queue is not unique - starting with any other node or using a diff^t ordering of neighbours → produces diff^t spanning trees. (same as DFS)

Precedence relationship → DAG Determination / Topological Sort

↳ Find strongly connected components

→ DAGs: strict partial order. Reflexive, Antisymmetric, Transitive

→ This ordering implies that for finite graphs, there'll be:
- some first elements corresponding to nodes with no in-edges
- " last " " " " " out edges

→ Topological ordering: When some tasks must be performed before others

→ $\delta(u,v)$: layer number of v in BFS rooted at u.

length of shortest path from u to v.

- Upper bound: $T(n) = O(f(n))$ ($\le c f(n)$) ; lower bound = Ω , ??
- Transitivity: $f = O(g)$, $g = O(h) \Rightarrow f = O(h)$
- Additivity: $f = O(h)$, $g = O(h) \Rightarrow f + g = O(h)$
- Algorithm → tractable - running time : polynomial.
- even matrix initialization : $O(n^2)$
- $i += \sqrt{n}$ → $\sqrt{n}\, t = n$ (total iterations)
- $i *= 8$ → $8^t = n$
- There are cases where functions with worse asymptotic behaviour performs better on small I/Ps.
- Asymptotic runtime behaviour is only guaranteed to hold for sufficiently large I/P size n.
- CAREFUL: for $y = 1$ to n do → if $y^2 < n$ → $O(\sqrt{n})$
- Binary Search - $O(\log n)$: Recursive Halving (RH)

  Exponential Search - $O(\log i)$ for index i → Recursive doubling (RD)

  Merge sort = $O(n \log n)$ → both RH & RD
- We frequently separate < Edge count - m → Tech ⇒ $m = \Omega(n^2)$; Node count - n

But some algo have runtime which is independent of node count, dependent on choice of graph rep^n, aided by sparse graphs or subgraphs.

- For I/Ps of sufficiently large size, runtime of A ~ some multiple of runtime of B.
- Adjacency list, Bipartiteness testing → $\Theta(m+n)$
- " Matrix, top. sort → $\Theta(n^2)$

- Input - domain (image) ; O/P → codomain (pre-image) / range
- $b^{\log_b(x)} = x$
- Shifting left <<- multip by b

  " right >> integer division by b
- Rule - Multiplication $(an+b)$ → $C \cdot a^k + D$
  - addition $(n+c)$ → not $k \cdot c$
  - mul + add → $A \cdot a^k + B$
  - subtraction → binary, power of 2
  - squares / doubles → 2^{2^k} (exponential)

- ~~DP (small I/P space)~~
- Input drawn from < Small: DP I/P space & generally large: recursion

  seek to recursively divide by constant factor

  reduce I/P size by const. subtraction : DP
- DP: Deep Recursion Tree → Exponential, bottom up

  D&C: Shallow " " [Greedy : Top down]
- Greedy: Rule is selected → ↓ I/P size < select / reject some portion of I/P

- Depends on best available choice at the moment and then solves subproblems that remain.

  [DP → solves subproblems before making a choice]
- 0-1 Knapsack ✗ Greedy → fractional knapsack

  Calculate value density (v_i/w_i per item) → sort ascending order → greedy ✓ → highest rem-value.
- Dijkstra → SSSP, min priority queue, non-negative edge.

  → Inserting a value ($d(u) = \infty$)
  → Extracting min value
  → Updating dist (change key)

  } each requires $O(\log n)$ time

  $O(m \log n)$ (last loop → $O(m)$)]
- Prim's → MST, undirected only, -ve edges
- Diff b/w D & P: starting node s → not fundamental, comparison against & value reassignment → not cumulative
- Proving < Diff: feasibility : Does algo produce correct sol^n. ; optimality : Is algo no worse than any other sol^n w.r.t its objective.
- Unweighted Interval Scheduling → If two intervals overlap in time, then they conflict & both can't be scheduled. (Mutually compatible)

  Options → earliest available starting time ; smallest available interval ; fewest conflicts ; $\boxed{\text{Earliest finishing time}}$ → Optimal (ascending finish times) comparison ↑ basic sorting $O(n \log n)$

  Greedy Algo Stays Ahead

  $[s(i) < s(j) < f(i)$ or $s(i) < f(j) < f(i)$.

  → local optimal decisions lead to overall globally optimal algo
- Deadline Interval Scheduling → Shortest Time Interval ; Smallest Slack Time ; $\boxed{\text{Earliest remaining deadline}}$

  Max^m lateness is max of all values.

→ All schedules with no inversions and no idle time have same maximum lateness.

→ Will schedule all jobs regardless of lateness.